



NATURAL LANGUAGE PROCESSING

Project Title

SMS SPAM DETECTION USING NATURAL LANGUAGE PROCESSING AND MACHINE LEARNING

Programme	BBA (Artificial Intelligence)
Academic Year	2026-27
Team Members	1. Sarvesh Wani - 25030422140 2. Sanando Maitra - 25030422105 3. Priyanshu Akshat - 25030422095
Faculty	Dr. Nabanita Choudhury, Assistant Professor

SYMBIOSIS ARTIFICIAL INTELLIGENCE INSTITUTE (SAII)

Constituent of Symbiosis International (Deemed University)

Lavale, Pune - 412115

DECLARATION

We hereby declare that the Project Report entitled "**SMS Spam Detection Using Natural Language Processing and Machine Learning**", which is being submitted to **Symbiosis Artificial Intelligence Institute (SAII), Symbiosis International (Deemed University)**, in partial fulfilment of the requirements for the completion of the third semester of the **Bachelor of Business Administration (Artificial Intelligence)**, is a bonafide report of the work carried out by us. The material contained in this report has not been submitted to any other university or institution.

Place: SAI, SIU, Pune

Date: 10/10/2026

Sarvesh Wani (PRN: 25030422140)

Sanando Maitra (PRN: 25030422105)

Priyanshu Akshat (PRN: 25030422095)

CERTIFICATE

This is to certify that the Project Report entitled "**SMS Spam Detection Using Natural Language Processing and Machine Learning**", submitted by Mr. Sarvesh Wani (PRN: 25030422140), Mr. Sanando Maitra (PRN: 25030422105) and Mr. Priyanshu Akshat (PRN: 25030422095) as a record of the work carried out by them, is accepted as a report submission in partial fulfilment of the requirements for the completion of the third semester of the **Bachelor of Business Administration (Artificial Intelligence)**.

Project Guide

Dr. Nabanita Choudhury

Assistant Professor, Course Instructor

Symbiosis Artificial Intelligence Institute

TABLE OF CONTENTS

1. Abstract	7
2. Introduction.....	8
3. Problem Statement and Objectives	9
4. Dataset Description.....	10
5. Data Preprocessing.....	14
6. Feature Representation / Embeddings	16
7. Proposed Methodology / System Architecture	18
8. NLP Model / Technique Used	20
9. Implementation	21
10. Results and Evaluation.....	23
11. Error Analysis / NLP Failure Analysis	27
12. Ethical Considerations and Limitations	29
13. Conclusion	30
14. Future Scope	30
15. References.....	32
16. GitHub Repository.....	32
17. Team Contribution.....	33
18. Annexure.....	34
19. Audit Sheet.....	35

LIST OF TABLES

Table 1. Dataset structure used in the project.	10
Table 2. Dataset cleaning summary.	11
Table 3. Class distribution after deduplication.	11
Table 4. Average message length by class.	12
Table 5. Examples of the preprocessing output.	15
Table 6. Train-test and TF-IDF configuration.	16
Table 7. Machine learning models and their settings (random_state=42 for all models that use it).	20
Table 8. Programming language and key libraries.	21
Table 9. Predictions on manually supplied new messages.	22
Table 10. Model performance comparison on the test set (spam as the positive class).	23
Table 11. Detailed classification report for Linear SVM.	25
Table 12. Error counts from the confusion matrices and derived error rates (test set: 903 ham, 131 spam).	27
Table 13. Failure cases visible in the preprocessing and prediction outputs.	27
Table 14. Contribution of each team member.	33

LIST OF FIGURES

Figure 1. Class distribution of ham and spam messages.	11
Figure 2. Message length distributions (characters and words) by class.	12
Figure 3. Top 20 words in spam and ham messages after cleaning.	13
Figure 4. Spam and ham word clouds.	13
Figure 5. End-to-end workflow of the SMS spam detection system.	18
Figure 6. Output of the new-message test in the implementation.	22
Figure 7. Model comparison across accuracy, precision, recall, F1-score and ROC-AUC. ...	23
Figure 8. Confusion matrices for the four classifiers (Naive Bayes, Logistic Regression, Linear SVM, Random Forest).	24
Figure 9. Precision-Recall curves for model comparison.	25
Figure 10. Most influential Logistic Regression features for spam and ham.	26

1. Abstract

One of the typical text classification problems is the problem of spam SMS detection, where promotional or malicious messages must be detected and distinguished from the legitimate ones (ham). In this research project, we implemented the full NLP pipeline for detecting SMS spam based on the UCI SMS Spam Collection. This dataset includes 5,572 real-life messages in UK English. After data cleaning and removal of 403 duplicated messages, there were 5,169 unique messages left – 4,516 ham messages and 653 spam messages, thus an imbalance of about 6.9:1.

The text preprocessing included the conversion of text to lowercase, URL, currency symbol and number normalization, removal of punctuation, text tokenization, removal of stopwords and lemmatization. The resulting text was then encoded into TF-IDF features with unigrams and bigrams with top 3,000 features selected. For comparison purposes, four different classifiers have been implemented – Multinomial Naive Bayes, Logistic Regression, Linear SVM and Random Forest. Weighting of classes has been performed for the models which allow such procedure. Evaluation was performed using accuracy, precision, recall, F1-score and ROC-AUC on stratified test sample of 1,034 messages.

The highest performance was obtained using linear SVM with an accuracy rate of 98.26%, a precision rate of 94.49%, a recall rate of 91.60%, an F1-score of 93.02% and an ROC-AUC of 0.9939. All three manual SMS spam messages were correctly classified using the final pipeline. In general, the project demonstrated that the classical NLP pipeline based on TF-IDF and machine learning methods can be highly effective when detecting SMS spam.

2. Introduction

2.1 Background and Context

SMS remains a familiar and convenient way to communicate, but it is also used to send unwanted advertisements, fake offers, prize claims and other misleading messages. Automatically identifying these messages is therefore a useful text-classification task. Each SMS is classified as either legitimate (ham) or spam.

2.2 Motivation for Choosing the Problem

Checking messages one by one would take time and would be difficult to manage at scale. An automated classifier can provide a useful first layer of screening. We chose this problem because it demonstrates a practical NLP workflow: raw text must be cleaned and converted into a useful representation before a machine learning model can make a prediction. The UCI dataset is also widely used in academic work, making the results easier to compare with other experiments.

2.3 Role of NLP in the Proposed Solution

In this project, NLP turns raw SMS text into information that a machine learning model can use. We clean the messages through lowercasing, normalization, tokenization, stopword removal and lemmatization, and then convert them into numerical TF-IDF features. The classifiers learn from these features and use the patterns they identify to distinguish legitimate messages from spam.

2.4 Scope and Applications

This project covers the binary-classification workflow implemented in our Python project on Google Colab. It includes data inspection, exploratory analysis, text preprocessing, TF-IDF vectorization, traditional machine learning, model evaluation, interpretation and testing with new messages. Transformer models and deep learning are outside the scope of the current implementation. The approach could be useful for:

- Message filtering and spam screening.
- Communication platforms that need an initial spam classification layer.
- Fraud-awareness systems that flag suspicious promotional messages.
- Academic demonstrations of NLP text classification.
- Foundational components for larger messaging-security systems.

3. Problem Statement and Objectives

3.1 Problem Statement

The goal of this project is to automatically sort SMS messages into two categories: ham (legitimate) and spam. To do this reliably, the system must account for informal language, punctuation, URLs, currency symbols, numbers and common words. It must also handle the imbalance between ham and spam messages and keep the training and test data separate so that the evaluation remains fair.

3.2 Objectives

1. Examine and prepare the UCI SMS Spam Collection dataset.
2. Eliminate duplicates before training models
3. Class distribution and important attributes of ham and spam text messages should be studied.
4. Develop a uniform pipeline for preprocessing texts.
5. Messages should be transformed to TF-IDF unigrams and bigrams.
6. Model the data using Multinomial Naive Bayes, Logistic regression, Linear SVM and Random Forest classifiers.
7. Models should be compared using accuracy, precision, recall, F1-score, ROC-AUC and other measures.
8. Validate the built pipeline using new SMS messages.

4. Dataset Description

4.1 Dataset Name, Source and Overview

For this project, we used the SMSSpamCollection dataset, commonly known as the UCI SMS Spam Collection, from the UCI Machine Learning Repository (<https://archive.ics.uci.edu/dataset/228/sms+spam+collection>). It contains 5,572 real SMS messages written in UK English. Each record includes a label indicating ham or spam and the original message text. The source file is tab-separated and does not have a header row.

The messages use a wide range of words and cover more than one type of SMS. This variety is helpful because a model trained on different examples of ham and spam has a better chance of handling unfamiliar wording than a model trained on a narrow set of messages.

4.2 Dataset Structure

Table 1. *Dataset structure used in the project.*

Column	Description	Type
label	Original class label: ham or spam	Object
message	Raw SMS message text	Object
spam	Binary representation: 1 for spam and 0 for ham	Integer

4.3 Data Loading and Inspection

We load the file with pandas, using a tab character to separate the two fields. Because the source file has no header row, we name the columns label and message. We then add a third column, spam, and convert the text label into a binary value.

```
df = pd.read_csv('SMSSpamCollection', sep='\t', header=None,
                 names=['label', 'message'])
df['spam'] = (df['label'] == 'spam').astype(int)
```

The initial DataFrame contains 5,572 rows and three columns. Our inspection found 5,572 non-null values in each column, so there are no missing values in label, message or spam.

4.4 Duplicate Removal

There are 403 duplicate rows in total, and all the rows have duplicate messages. We first drop the duplicate messages and then divide the data into a training and testing set. This is important since if the same message appears in both the training and testing sets, it would overstate the accuracy of the model.

Table 2. *Dataset cleaning summary.*

Stage	Rows / Records
Initial dataset	5,572
Duplicate messages identified	403
Dataset after deduplication	5,169
Missing values	0

4.5 Class Distribution

Once the duplicates have been eliminated, there are 5,169 total messages in the dataset, including 4,516 ham and 653 spam. That is to say, roughly 87.4% of the total messages are classified as ham while the rest, 12.6%, are spam messages.

Table 3. *Class distribution after deduplication.*

Class	Count	Proportion
Ham (0)	4,516	87.4%
Spam (1)	653	12.6%
Total	5,169	100%

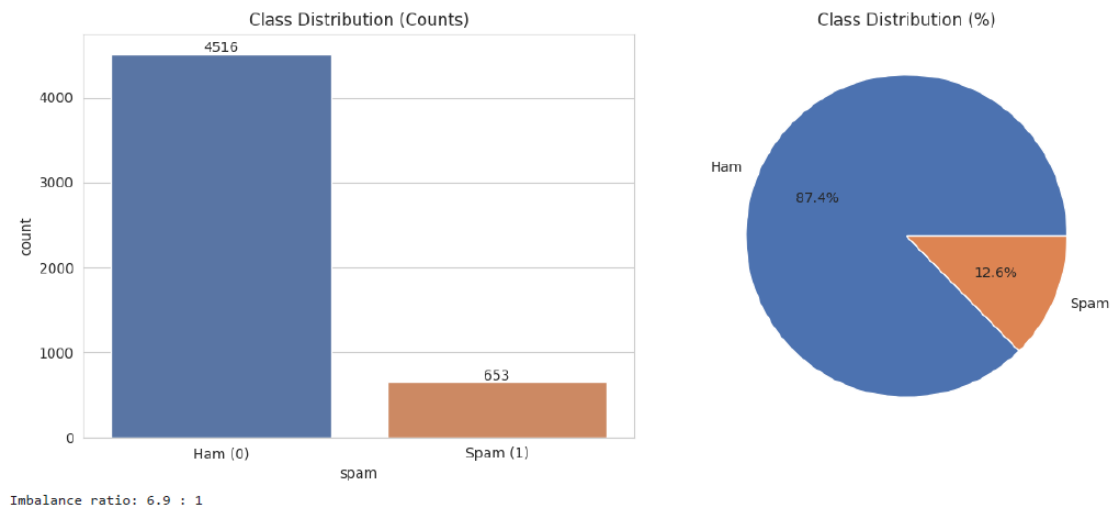


Figure 1. *Class distribution of ham and spam messages.*

The chart makes the imbalance clear: legitimate messages account for most of the dataset, while spam forms the smaller class. A model could still achieve high accuracy by predicting ham most of the time, even if it missed many spam messages. To help address this issue, we use `class_weight='balanced'` for Logistic Regression, Linear SVM and Random Forest, while keeping Multinomial Naive Bayes as an unweighted baseline. For this reason, we consider precision, recall, F1-score and ROC-AUC alongside accuracy.

4.6 Message Length Analysis

The number of characters and words per message were used to evaluate the messages. It is clear that the average values for each class are different. A ham message has an average value of 70.91 characters and 14.24 words, whereas a spam message has 137.70 characters and 23.74 words.

Table 4. Average message length by class.

Class	Average characters	Average words
Ham	70.905890	14.239814
Spam	137.704441	23.739663

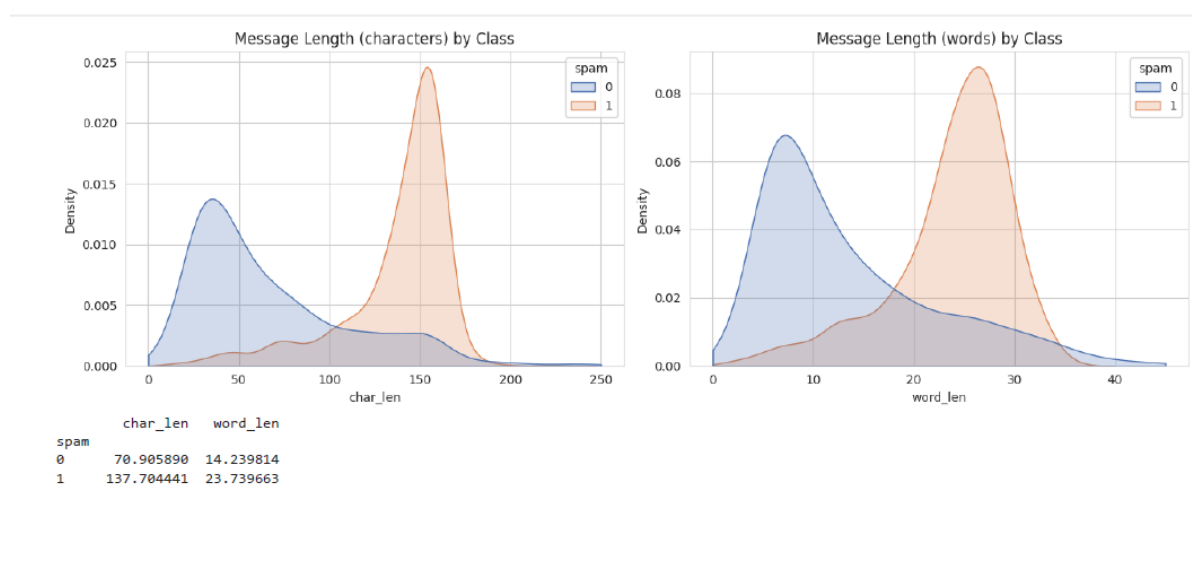


Figure 2. Message length distributions (characters and words) by class.

From the density graphs, we can see that spam emails have longer lengths as compared to ham emails. One possible reason could be the presence of some sort of offer and call to action within the spam emails, whereas ham emails are generally short and conversational.

4.7 Frequent Words

In addition to the above, we also compared the 20 most frequent words from each class after cleansing the text. Spam contains many words related to offers, prizes, calls, claims and urgency while ham contains many common conversational words.

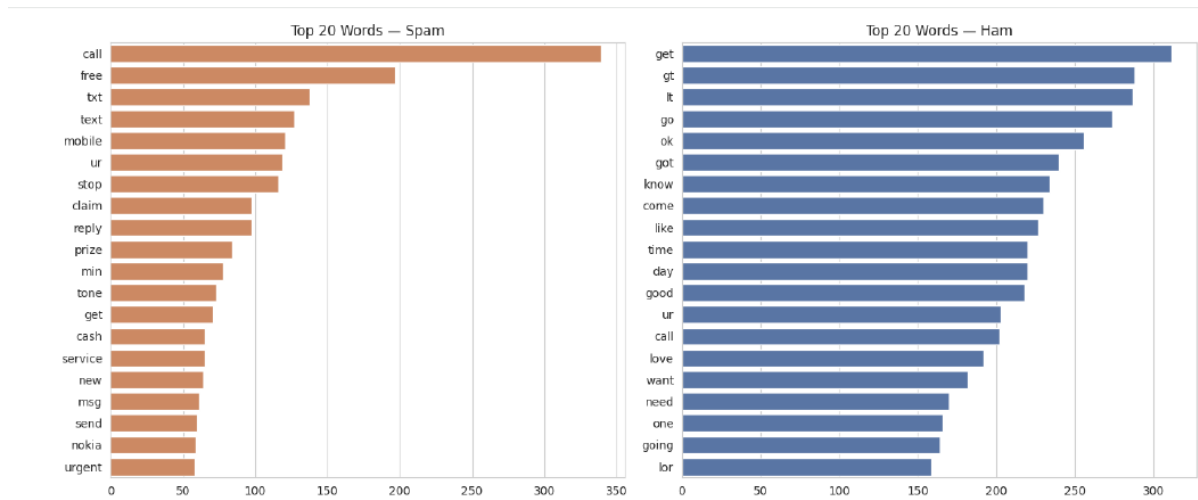


Figure 3. Top 20 words in spam and ham messages after cleaning.

4.8 Word Clouds

The word clouds offer a fast visual way to compare the vocabulary of both classes. The spam word cloud highlights words like free, win, prize, call, claim, text, and urgent. The ham word cloud has more conversational words. The word clouds can be used to analyze the data, but the classifier does not use just words; it uses the TF-IDF representation of each message.



Figure 4. Spam and ham word clouds.

5. Data Preprocessing

In order to extract features, we need to preprocess the original messages in order to make them more uniform in format. These tasks are performed with a function `clean_text()`, which was written specifically for our dataset. The main pre-processing actions and reasons for performing them are listed below.

5.1 Lowercasing

We make all messages lowercase in order to avoid having the model consider words like FREE and free to be different terms.

5.2 URL, Currency and Number Normalization

Links starting with either 'http' or 'www' are substituted with the token URL. The pound and dollar currency symbols are replaced with the CURRENCY token. Number sequences, consisting of four or more digits, are substituted with the token NUM. This approach is intended to save the general idea about a message including a link, currency amount or phone number, without preserving its exact value.

5.3 Punctuation and Non-Alphabetical Symbols

We delete any characters except lowercased letters and spaces, and we reduce multiple spaces to one.

5.4 Tokenization

We use NLTK's `word_tokenize` function to tokenize the pre-processed messages. The rest of the pre-processing steps will work not on strings but on tokens.

5.5 Stop-word Removal

We remove common English stopwords with NLTK's stopword list and discard tokens that contain only one character. These items usually provide little help in deciding whether a message is ham or spam.

5.6 Lemmatization

We use WordNetLemmatizer to bring words closer to their base forms. For example, receiving can be reduced to receive. Grouping related forms in this way helps limit unnecessary variation in the text.

5.7 Cleaning Function

```
def clean_text(text):
    text = str(text).lower()
    text = re.sub(r'http\S+|www\S+', ' URL ', text)
    text = re.sub(r'£|\$', ' CURRENCY ', text)
    text = re.sub(r'\b\d{4,}\b', ' NUM ', text)
    text = re.sub(r'^a-z\s', ' ', text)
    text = re.sub(r'\s+', ' ', text).strip()
    tokens = word_tokenize(text)
    tokens = [lemmatizer.lemmatize(t) for t in tokens
              if t not in stop_words and len(t) > 1]
    return ' '.join(tokens)
```

5.8 Examples of Cleaned Messages

Table 5. *Examples of the preprocessing output.*

Original message (shortened)	Cleaned message (shown in output)
Go until jurong point, crazy.. Available only ...	go jurong point crazy available bugis great wo...
Ok lar... Joking wif u oni...	ok lar joking wif oni
Free entry in 2 a wkly comp to win FA Cup fina...	free entry wkly comp win fa cup final tkts st ...
U dun say so early hor... U c already then say...	dun say early hor already say

After applying the cleaning function, we check whether any messages have been left with no words. In this dataset, 12 messages become empty. This check is useful because aggressive cleaning can remove all meaningful tokens from a very short message.

One issue remains with the placeholder tokens. URL, CURRENCY and NUM are first inserted in uppercase, but the next cleaning step keeps only lowercase letters, so the placeholders are removed before tokenization. As a result, values such as £100,000 and 81010 do not remain in the cleaned text. A link such as bit.ly/claim123 also does not match the current pattern, which only checks for http or www, so it is reduced to words such as “bit ly claim”. These limitations are revisited in Section 11.

6. Feature Representation / Embeddings

6.1 Need for Numerical Representation

Machine learning models cannot work directly with raw text strings, so we first need to turn the cleaned messages into numerical vectors. We use Term Frequency-Inverse Document Frequency (TF-IDF), a widely used method for representing text in classification tasks.

6.2 TF-IDF and the Reason for Choosing It

TF-IDF assigns more weight to words that are useful in a particular message but less common across the full collection. This is helpful for spam detection because terms such as claim, prize and txt may be informative, while very common words should carry less weight. TF-IDF produces sparse, high-dimensional vectors that work well with linear models such as Logistic Regression and Linear SVM. Its feature weights can also be inspected to see which words influence a prediction.

6.3 Unigrams and Bigrams

We set `ngram_range=(1,2)`, which means the vectorizer considers both individual words (unigrams) and pairs of words (bigrams). Bigrams allow the model to learn from short phrases that may carry more meaning than a single word by itself.

6.4 Train-Test Split

We split the cleaned data into training and test sets using an 80:20 ratio. Stratification keeps the proportion of ham and spam similar in both sets, and setting `random_state` to 42 makes the split reproducible. The split produces 4,135 training messages and 1,034 test messages; the test set contains 903 ham messages and 131 spam messages.

6.5 TF-IDF Configuration

Table 6. *Train-test and TF-IDF configuration.*

Parameter	Value	Purpose
test_size	0.20	20% of the data is used for testing
stratify	spam labels	Maintains class proportions
random_state	42	Reproducible split
max_features	3000	Keeps the top 3,000 features
ngram_range	(1,2)	Uses unigrams and bigrams
min_df	2	Keeps terms appearing in at least two documents

```
tfidf = TfidfVectorizer(max_features=3000, ngram_range=(1,2), min_df=2)
X_train = tfidf.fit_transform(X_train_text)  # fitted on training text only
X_test = tfidf.transform(X_test_text)
```

The training matrix contains 4,135 samples and 3,000 features, while the test matrix contains 1,034 samples with the same number of features. Importantly, we fit TF-IDF only on the

training messages and then apply the fitted vectorizer to the test messages. This prevents information from the test vocabulary from leaking into model training.

7. Proposed Methodology / System Architecture

7.1 End-to-End Workflow

The workflow begins with raw SMS messages and moves through cleaning and normalization, tokenization, stopwords removal, lemmatization, TF-IDF feature extraction and model training. The trained models are then evaluated and used to predict the labels of new messages. Figure 5 shows how these stages connect from input to output.

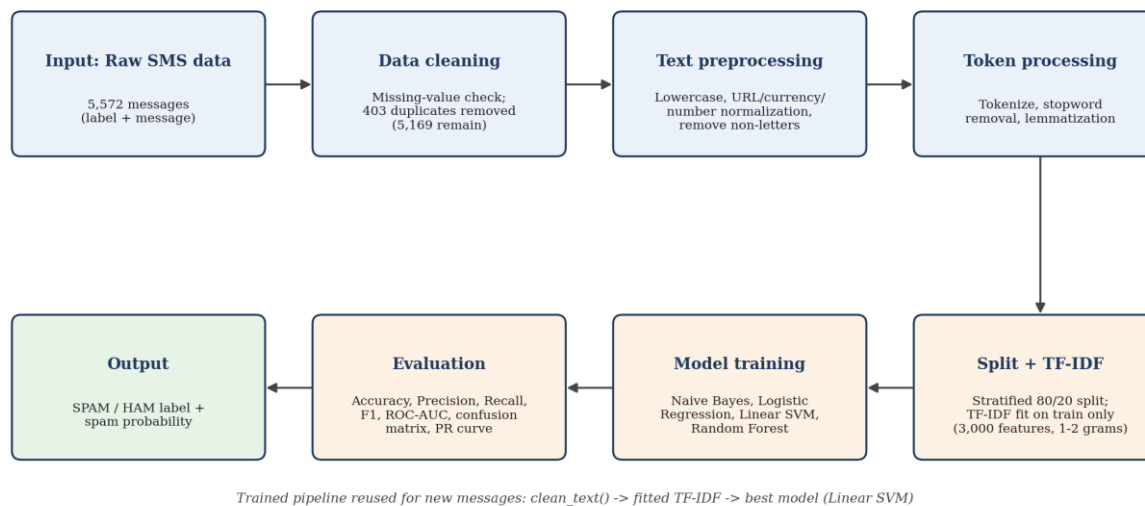


Figure 5. End-to-end workflow of the SMS spam detection system.

7.2 Stages of the Pipeline

1. **Data loading and inspection:** read the tab-separated file, name its columns and create a binary label for spam.
2. **Data cleaning:** check for missing values and remove the 403 duplicate messages.
3. **Exploratory analysis:** review the class balance, message lengths and most frequent words.
4. **Text preprocessing:** lowercase the messages, normalize selected values, remove punctuation, tokenize the text, remove stopwords and lemmatize the words.
5. **Split and vectorization:** create a stratified 80:20 train-test split and fit TF-IDF only on the training text.
6. **Model training:** train all four classifiers using the same training matrix.
7. **Evaluation:** compare the metrics, confusion matrices, ROC curves and precision-recall curves on the held-out test set.
8. **Interpretation and testing:** examine influential features and run predictions on new messages.

7.3 Key Design Decisions

- **Leakage prevention:** remove duplicates before the train-test split and learn the TF-IDF vocabulary from training text only.
- **Imbalance handling:** use `class_weight='balanced'` for the models that support it, and assess precision, recall, F1-score and ROC-AUC rather than relying on accuracy alone.
- **Fair comparison:** give each model the same TF-IDF training matrix so that performance differences mainly reflect the algorithms, rather than different feature sets.

8. NLP Model / Technique Used

We tested four traditional machine learning algorithms rather than depending on a single classifier. Comparing them helps show which approach works best with the TF-IDF features prepared for this project.

8.1 Multinomial Naive Bayes

Multinomial Naive Bayes serves as a standard baseline for text classification. It works well with features based on word counts or frequency. Here, it remains an unweighted baseline because it does not support class weighting in the same way as the other selected models.

8.2 Logistic Regression

Logistic Regression is a linear classifier that estimates the probability of a message belonging to one of two classes. Its coefficients are also useful for interpretation because they show which features push the prediction toward spam or ham.

8.3 Linear Support Vector Machine

Linear SVM is a strong traditional choice for text data because it handles high-dimensional, sparse features well. We use LinearSVC with balanced class weights and wrap it in CalibratedClassifierCV so that probability estimates are available for evaluation and example predictions.

8.4 Random Forest

Random Forest combines predictions from multiple decision trees. We include it as a non-linear alternative, allowing us to compare its performance with the linear models used for text classification.

8.5 Model Configuration

Table 7. Machine learning models and their settings (*random_state=42 for all models that use it*).

Model	Configuration / Role
Multinomial Naive Bayes	Unweighted classical text-classification baseline
Logistic Regression	max_iter=1000; class_weight='balanced'
Linear SVM	LinearSVC with class_weight='balanced', calibrated probabilities
Random Forest	200 trees; class_weight='balanced'; n_jobs=-1

8.6 Training Procedure

Each classifier is trained on `X_train` and `y_train`. We then predict labels for `X_test` and retain probability estimates for ROC-AUC and precision-recall analysis. The fitted models, predictions and probabilities are stored in dictionaries, which lets us evaluate all four algorithms consistently.

9. Implementation

9.1 Programming Language and Libraries

Table 8. *Programming language and key libraries.*

Purpose	Tools
Language and environment	Python; Google Colab notebook
Data handling	pandas, NumPy, re, collections.Counter
Visualization	Matplotlib, Seaborn, WordCloud
NLP	NLTK (stopwords, punkt tokenizer, WordNet lemmatizer)
Machine learning and metrics	scikit-learn (TfidfVectorizer, MultinomialNB, LogisticRegression, LinearSVC, CalibratedClassifierCV, RandomForestClassifier, evaluation metrics)

9.2 Major Modules and Components

- Data loading, inspection and duplicate removal.
- Exploratory data analysis (class balance, message length, frequent words, word clouds).
- Text preprocessing function `clean_text()`.
- Train-test split and TF-IDF vectorization.
- Model definition and training loop for the four classifiers.
- Evaluation module (metrics table, confusion matrices, ROC and precision-recall curves, classification report).
- Interpretation module (Logistic Regression coefficients).
- Prediction function `predict_message()` for new messages.

9.3 Application or User Interface

We did not build a separate application or graphical user interface for this project. Instead, the system runs in a Google Colab notebook, where a new message can be classified by calling `predict_message()`. The submitted work does not include deployment as a real-time mobile app or messaging service.

9.4 Working System: Testing on New Messages

Once training and evaluation are complete, the `predict_message` function can process new input. It applies the same cleaning steps used during training, transforms the message with the fitted TF-IDF vectorizer, predicts a class with the selected model and returns the estimated spam probability.

```
def predict_message(text, model_name=None):
    model_name = model_name or best_model_name
    cleaned = clean_text(text)
    vec = tfidf.transform([cleaned])
    pred = fitted_models[model_name].predict(vec)[0]
    proba = fitted_models[model_name].predict_proba(vec)[0][1]
    label = 'SPAM' if pred == 1 else 'HAM'
```

Table 9. *Predictions on manually supplied new messages.*

Test message	Prediction	Reported spam probability
Congratulations! You have WON a FREE prize. Click here to claim now: bit.ly/claim123	SPAM	0.99
Hey, are we still meeting for lunch tomorrow at 1pm?	HAM	0.00
URGENT! You have won a 1 week FREE membership in our £100,000 Prize Jackpot! Txt WORD to 81010	SPAM	0.99

```
Message: Congratulations! You have WON a FREE prize. Click here to claim now: bit.ly/claim123
Cleaned: congratulation free prize click claim bit ly claim
Prediction: SPAM (spam probability: 0.99)

Message: Hey, are we still meeting for lunch tomorrow at 1pm?
Cleaned: hey still meeting lunch tomorrow pm
Prediction: HAM (spam probability: 0.00)

Message: URGENT! You have won a 1 week FREE membership in our £100,000 Prize Jackpot! Txt WORD to 81010
Cleaned: urgent week free membership prize jackpot txt word
Prediction: SPAM (spam probability: 0.99)
```

Figure 6. *Output of the new-message test in the implementation.*

After preprocessing, the first message is classified as spam because its features include terms such as congratulation, free, prize, click and claim. The second message is classified as ham and has a more conversational representation. The third message is also classified as spam, with terms linked to urgency, free membership, prizes and txt. These examples provide a quick qualitative check, but they do not replace evaluation on the held-out test set.

These tests also offer a small indication of how the model handles new wording. It identifies a typical prize-scam message with a high reported spam probability. At the same time, the example highlights how much a model's performance on unfamiliar messages depends on the range of language represented in its training data.

10. Results and Evaluation

10.1 Evaluation Metrics

We use several metrics to evaluate the models. Accuracy measures the share of all test messages classified correctly. Precision tells us how many messages predicted as spam really are spam, while recall measures how many of the actual spam messages are caught. The F1-score combines precision and recall into one measure, and ROC-AUC indicates how well the model separates the classes across different thresholds. Because spam is the minority class, precision and recall are especially important: a model may have high accuracy yet still miss many spam messages if its recall is low.

10.2 Model Performance Comparison

Table 10. *Model performance comparison on the test set (spam as the positive class).*

Model	Accuracy	Precision	Recall	F1-score	ROC-AUC
Linear SVM	0.982592	0.944882	0.916031	0.930233	0.993884
Random Forest	0.976789	0.981982	0.832061	0.900826	0.992442
Logistic Regression	0.970986	0.863309	0.916031	0.888889	0.991534
Multinomial Naive Bayes	0.974855	1.000000	0.801527	0.889831	0.986115

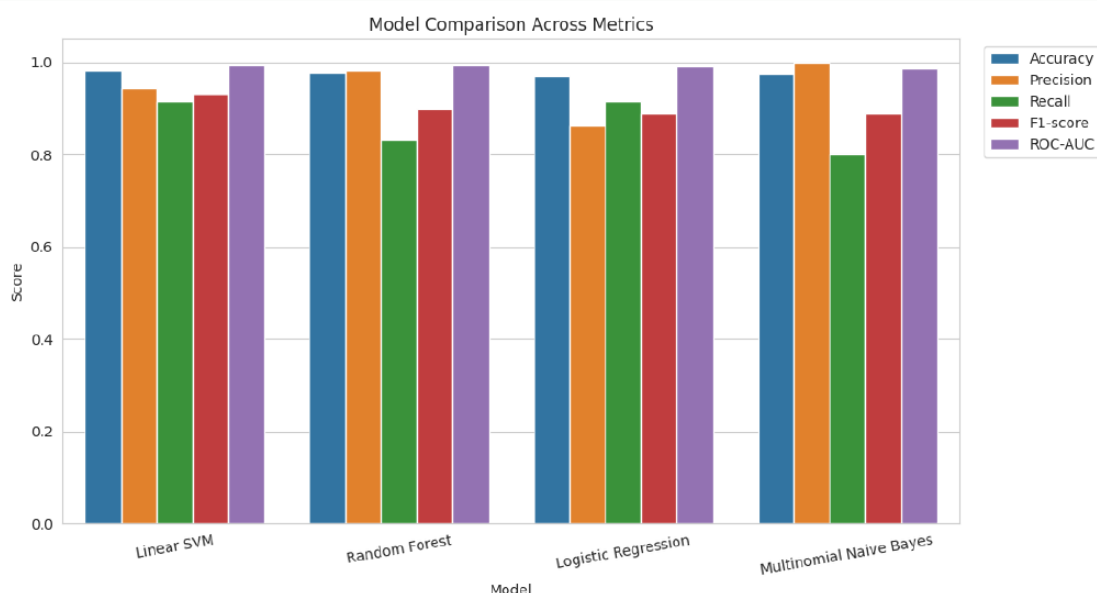


Figure 7. *Model comparison across accuracy, precision, recall, F1-score and ROC-AUC.*

Linear SVM achieves the highest accuracy, F1-score and ROC-AUC of the four models in this experiment, with an ROC-AUC of 0.993884. Random Forest has high precision at 0.981982, but its recall is lower at 0.832061. Multinomial Naive Bayes reaches a precision of 1.000000, meaning that none of the messages it flagged as spam were ham in this test set, but its recall is only 0.801527, so it misses more actual spam messages. Logistic Regression has

the same recall as Linear SVM (0.916031), but lower precision and F1-score. Based on the overall ROC-AUC comparison, Linear SVM is selected as the best-performing model.

10.3 Confusion Matrices

A confusion matrix shows four types of outcomes: true negatives, false positives, false negatives and true positives. Figure 8 displays the matrices for all four classifiers, making it easier to compare the errors each model makes.

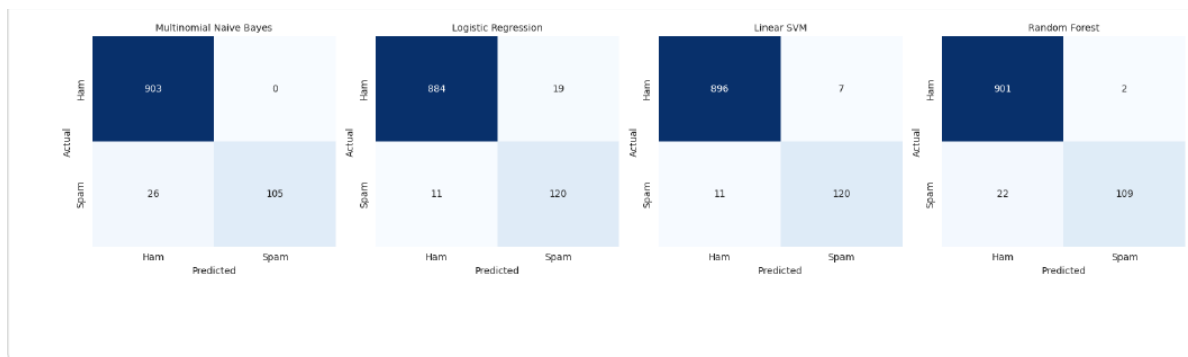


Figure 8. Confusion matrices for the four classifiers (Naive Bayes, Logistic Regression, Linear SVM, Random Forest).

For Linear SVM, 896 ham messages are correctly labelled as ham, while 7 legitimate messages are wrongly labelled as spam. Of the 131 spam messages, 120 are correctly identified and 11 are labelled as ham. These counts match the precision, recall and F1-score reported for the selected model.

10.4 ROC Curves

The ROC curves for all four models stay close to the upper-left corner, suggesting that each classifier separates ham and spam well. Their ROC-AUC values range from 0.986 to 0.994, with Linear SVM achieving the highest score.

10.5 Precision-Recall Curves

Precision-recall curves are particularly helpful when the classes are imbalanced because they show the balance between detecting spam and avoiding false alarms. Average precision is 0.97 for Linear SVM and Random Forest, and 0.95 for Logistic Regression and Multinomial Naive Bayes. The curves maintain high precision through much of the useful recall range.

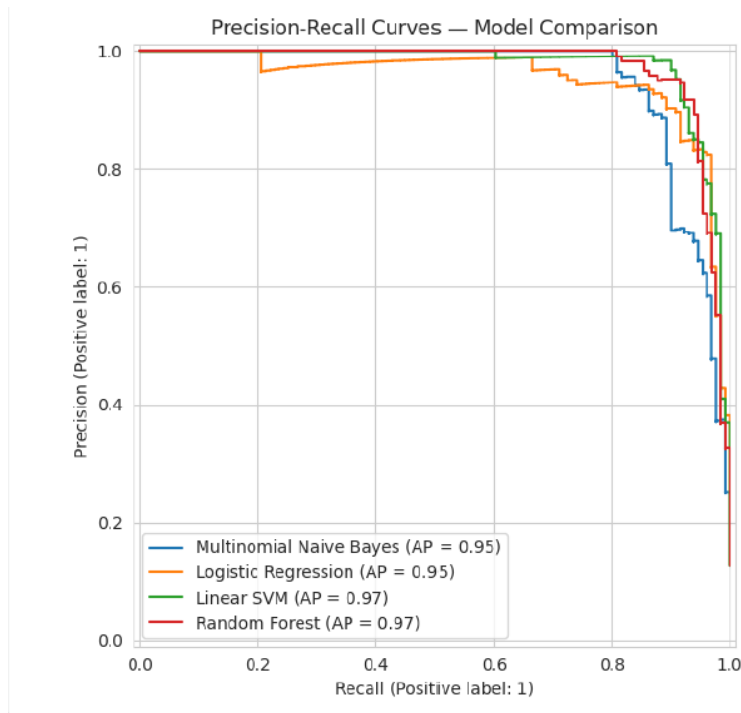


Figure 9. Precision-Recall curves for model comparison.

10.6 Detailed Classification Report: Linear SVM

Table 11. Detailed classification report for Linear SVM.

Class	Precision	Recall	F1-score	Support
Ham	0.99	0.99	0.99	903
Spam	0.94	0.92	0.93	131
Accuracy			0.98	1,034
Macro average	0.97	0.95	0.96	1,034
Weighted average	0.98	0.98	0.98	1,034

10.7 Feature Interpretation: Most Influential Words

We examined the Logistic Regression coefficients to identify features more strongly linked to each class. Positive coefficients push predictions toward spam, while negative coefficients push them toward ham. Stronger spam-associated terms include txt, call, mobile, free, reply, text, claim, prize, stop, service, tone, win, chat, new, cash, uk, per, com, video and ringtone. Terms more closely associated with ham include ok, gt, lt, home, later, da, lor, got, way, oh, still, say, going, much, lol, hey, sorry, good and wat.

The coefficient plot also connects the model results with the earlier word-frequency analysis.

Promotional language, prize claims and prompts to call or reply tend to push a message toward spam, whereas short, informal expressions tend to support the ham class. This does not mean that every message containing one of these terms is automatically assigned to that class; a term's contribution depends on the full TF-IDF vector and on how words appear together. The plot is best read as a summary of patterns learned from this dataset.

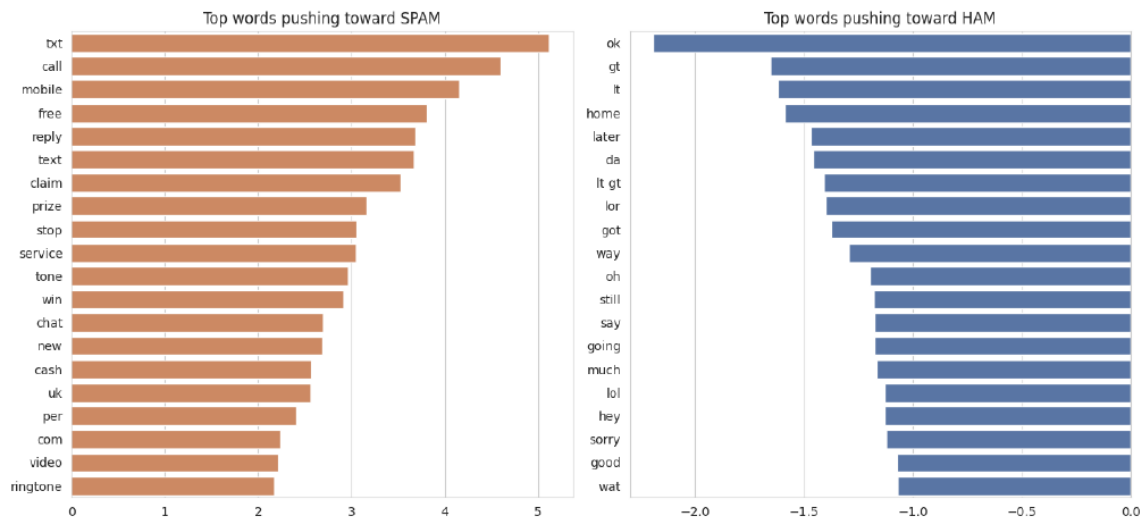


Figure 10. *Most influential Logistic Regression features for spam and ham.*

These words should not be treated as fixed rules for identifying spam or ham. The classifier considers the combined TF-IDF representation of the entire message, so a single word such as free or prize is not enough on its own to determine the class.

10.8 Discussion

- **Data quality and leakage prevention:** removing the 403 duplicate messages before the split reduces the chance that identical messages appear in both training and testing, which could otherwise make the evaluation look better than it really is.
- **Effect of class imbalance:** because the class ratio is about 6.9:1, accuracy by itself is not enough to judge the spam detector. Using class weighting for three models and examining precision, recall, F1-score and ROC-AUC gives a more useful picture of performance.
- **Model selection:** Linear SVM is selected as the best model based on ROC-AUC. Its confusion matrix contains only seven false-positive ham messages and eleven spam messages that were missed.
- **Practical meaning:** the results show that traditional machine learning can perform strongly on this dataset when the text is prepared carefully. At the same time, very high precision does not automatically mean that most spam is detected, and high accuracy can be misleading when the positive class is relatively small.

11. Error Analysis / NLP Failure Analysis

The notebook summarizes model errors with confusion matrices rather than listing every misclassified message. The discussion below therefore brings together the error counts and the specific issues visible in the preprocessing and prediction outputs.

11.1 Error Summary by Model

Table 12. Error counts from the confusion matrices and derived error rates (test set: 903 ham, 131 spam).

Model	False positives (ham flagged as spam)	False negatives (spam missed)	Total errors	Ham wrongly blocked (of 903)	Spam missed (of 131)
Linear SVM	7	11	18	0.78%	8.40%
Random Forest	2	22	24	0.22%	16.79%
Multinomial Naive Bayes	0	26	26	0.00%	19.85%
Logistic Regression	19	11	30	2.10%	8.40%

Linear SVM makes the fewest mistakes overall, with 18 errors among the 1,034 test messages. Logistic Regression has the most false positives, incorrectly flagging 19 legitimate messages, which is consistent with its lower precision of 0.863. Naive Bayes and Random Forest are more cautious about flagging ham, but they miss 26 and 22 of the 131 spam messages respectively. The two types of errors have different consequences: a false positive can hide a genuine message, while a false negative allows spam to reach the user.

11.2 Failure Cases Observed in the Pipeline

Table 13. Failure cases visible in the preprocessing and prediction outputs.

Case	Observed output	Issue and likely reason
Link without http/www	bit.ly/claim123 is cleaned to "bit ly claim"	The URL pattern only matches http or www, so the link is split into ordinary words and the link signal is lost.
Amount and shortcode	£100,000 and 81010 do not appear in "urgent week free membership prize jackpot txt word"	Uppercase placeholders (CURRENCY, NUM) are deleted by the next step, which keeps only lowercase letters.
Numbers inside words	"1pm" becomes "pm"	Digits are removed, so time and quantity information is lost.
Empty messages	12 messages contain no tokens after cleaning	Messages made only of stopwords, numbers, symbols or single characters carry no features, so the classifier has nothing to learn from.

11.3 Discussion of Failure Types

- **Informal language and spelling variation:** SMS messages use forms such as wif, oni, dun, hor, lor and wat. The standard English stopword list and lemmatizer do not convert

these forms into standard English. Some tokens, including lor, da and wat, still help identify ham, but terms appearing in fewer than two training messages are excluded because `min_df=2`.

- **Context dependence and ambiguity:** words such as call, get and ur occur in both classes. Because the bag-of-words representation captures word order only through bigrams, spam that avoids common promotional wording can resemble a legitimate message. This may account for some of the spam messages the models miss.
- **Encoding artifacts:** tokens such as gt and lt appear among the stronger ham features. They seem to originate from encoded characters in the source text rather than meaningful words, so the model may be relying on formatting noise instead of genuine language patterns.
- **Information lost during preprocessing:** Table 13 shows that link, currency and number details do not survive in the final cleaned text, even though these details could provide useful clues when detecting spam.

12. Ethical Considerations and Limitations

12.1 Ethical Considerations

- **Privacy:** the dataset contains real SMS messages, which are personal communications. A deployed version of this type of filter would also handle private messages, so it would need appropriate consent, strong security measures and minimal data retention.
- **Harm from false positives:** labelling a genuine message as spam may cause someone to miss important information. The best-performing model produces 7 false positives, compared with 19 for Logistic Regression. This is a practical concern, and the decision threshold should reflect how the system will be used.
- **Dataset representativeness and bias:** the collection contains UK English messages from the mid-2000s. It does not cover every region, language, slang style or modern spam technique, so performance may differ for other users and newer messaging environments.

12.2 Limitations

- The project uses TF-IDF and traditional machine learning rather than newer transformer-based language models.
- Because the dataset contains UK English SMS messages from the mid-2000s, it may not reflect current slang, vocabulary or spam tactics.
- The classes are imbalanced, so precision, recall and F1-score are more informative than accuracy on its own.
- The manually supplied examples are only a small sanity check; they are not a statistically meaningful external validation set.
- The model was trained for the ham-versus-spam task represented in this dataset, so its results may not transfer equally well to other languages, regions, communication channels or current messaging services.
- The results come from one train-test split. Since no cross-validation or hyperparameter tuning was reported, small differences between models should be interpreted cautiously, especially because the test set contains only 131 spam messages.
- The URL, currency and number placeholders are removed during the cleaning process described in Section 5, so the models do not ultimately use those signals.
- The project does not include a real-time mobile or messaging deployment. Because spam language changes over time, a production system would also need to be retrained regularly on newer messages.

13. Conclusion

This project built an end-to-end Natural Language Processing and machine learning workflow to detect SMS spam using the UCI SMS Spam Collection. The dataset began with 5,572 messages; after removing 403 duplicates, 5,169 unique records remained. The exploratory analysis also revealed a clear class imbalance: about 87.4% of the messages were ham and 12.6% were spam.

During preprocessing, the messages are converted to lowercase, selected URLs, currency symbols and long numbers are normalized, punctuation is removed, and tokenization, stopword removal and lemmatization are applied. TF-IDF then represents the cleaned text with 3,000 features using unigrams and bigrams. We trained and compared four traditional classifiers using several evaluation metrics.

Of the models tested, Linear SVM gives the strongest overall result based on ROC-AUC. It reaches 98.26% accuracy, 94.49% precision, 91.60% recall, a 93.02% F1-score and an ROC-AUC of 0.993884. Its confusion matrix and classification report also show strong performance on both ham and spam. All three manually supplied messages were classified as expected. Overall, the project met its stated objectives: we prepared the dataset, built a preprocessing and TF-IDF pipeline, trained and compared four models, and tested the final pipeline on new messages.

A key lesson from this work is that careful data handling can matter just as much as the choice of model. Removing duplicates before the split, fitting TF-IDF only on training data and accounting for class imbalance all make the evaluation more reliable. However, the age of the dataset and changes in spam language show that future work would benefit from newer data, regular retraining and possibly more advanced language models.

14. Future Scope

The improvements listed below were not part of the submitted pipeline, but they could be considered in a future version of the project.

- Handle placeholder tokens more carefully, for example by using lowercase placeholders, and recognize links such as bit.ly so that link, amount and shortcode clues are retained.
- Review the messages that were misclassified, then use cross-validation and hyperparameter tuning to make the model comparison more reliable.
- Explore word-embedding methods such as Word2Vec or GloVe to represent relationships between words.
- Test transformer-based models such as BERT to capture more context from each message.

- Use a larger, more recent SMS dataset to reflect current spam wording and tactics, and retrain the model regularly if it is used in practice.
- Add multilingual data to explore whether the classifier can handle messages in different languages.
- Experiment with different classification thresholds so that the balance between false positives and false negatives can be adjusted for the intended use.
- Connect the trained model to a real-time message-screening application or API.

15. References

- [1] Almeida, T. A., Gomez Hidalgo, J. M., & Yamakami, A. (2011). Contributions to the study of SMS spam filtering: New collection and results. Proceedings of the 11th ACM Symposium on Document Engineering (DocEng '11), 259-262.
- [2] UCI Machine Learning Repository. (n.d.). SMS Spam Collection dataset. <https://archive.ics.uci.edu/dataset/228/sms+spam+collection>
- [3] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830. (scikit-learn documentation: <https://scikit-learn.org>)
- [4] Bird, S., Klein, E., & Loper, E. (2009). Natural Language Processing with Python. O'Reilly Media. (NLTK documentation: <https://www.nltk.org>)
- [5] pandas, NumPy, Matplotlib, Seaborn and WordCloud library documentation, as used in the Python/Google Colab implementation.

16. GitHub Repository

The full implementation is maintained in the project repository. It includes the Google Colab notebook, the dataset reference and instructions for reproducing the results, rather than reproducing the entire code in this report.

- GitHub Repository Link: <https://github.com/Sarvesh-Wani-07/spam-detection-NLP>

17. Team Contribution

The table below summarizes the specific work completed by each team member.

Table 14. *Contribution of each team member.*

Team Member	PRN	Specific Contribution
Sarvesh Wani	25030422140	PowerPoint presentation and Report
Sanando Maitra	25030422105	PowerPoint presentation and Report
Priyanshu Akshat	25030422095	Coding And Audit Sheet

18. Annexure

Annexure A: Reproducibility Notes

- Random seed: `RANDOM_STATE = 42` is used for the train-test split and for all models that use randomness.
- NLTK resources required: `stopwords`, `punkt`, `punkt_tab`, `wordnet` and `omw-1.4`.
- Dataset file: `SMSSpamCollection` (tab-separated, no header row), loaded using `pandas`.
- Environment: Google Colab with `scikit-learn`, `NLTK`, `pandas`, `NumPy`, `Matplotlib`, `Seaborn` and `WordCloud`.

Annexure B: Evaluation Code Snippet

```
results.append({
    'Model': name,
    'Accuracy': accuracy_score(y_test, y_pred),
    'Precision': precision_score(y_test, y_pred),
    'Recall': recall_score(y_test, y_pred),
    'F1-score': f1_score(y_test, y_pred),
    'ROC-AUC': roc_auc_score(y_test, y_proba),
})
```

The full source code and implementation details are available in the GitHub repository listed in Section 16.

Audit Sheet

1. Model details

Field	Description
Model name	SMS Spam Lab: four classifiers (Naive Bayes, Logistic Regression, Linear SVM and Random Forest) plus a soft-voting ensemble
Task	Classifies English SMS messages into two categories: spam or ham
Model	TF-IDF (top 3,000 unigram and bigram features) with MultinomialNB, balanced LogisticRegression, calibrated balanced LinearSVC, and RandomForest (200 trees, balanced)
Training data	UCI SMS Spam Collection after removing 403 duplicates; 4,135 training messages; 12.6% spam
Test data	1,034 held-out messages, including 131 spam; stratified 80/20 split with random seed 42
Preprocessing	Text lowercasing; URL, currency and number placeholders; punctuation and stopword removal; WordNet lemmatisation
Threshold libraries	/ 0.50 (adjustable in the app); Python, scikit-learn, NLTK, Streamlit, Plotly

2. Intended use

- **Intended:** This project is designed as an educational example of a traditional NLP workflow. It can help sort English SMS messages for review, with a person checking any messages that are flagged.
- **Out of scope:** The system is not intended to block messages automatically without human review, handle other languages or mixed-language text, detect newer delivery or bank-link scams that are missing from the dataset, or support decisions about individual people.

3. Overall performance

Linear SVM performed best as an individual model (accuracy 0.982, F1 score 0.927 and the lowest Brier score, 0.015). The majority-class baseline, which labels every message as “ham,” reaches about 87.3% accuracy but does not catch any spam. Linear SVM improves on that baseline by about 10.9 percentage points and detects 92% of spam.

Model	Acc	Prec	Recall	F1	ROC-AUC	Brier	FP	FN
Majority-class baseline	~0.873	–	0.000	0.000	0.500	–	0	131
Multinomial Naive Bayes	0.975	1.000	0.802	0.890	0.986	0.022	0	26
Logistic Regression	0.971	0.863	0.916	0.889	0.991	0.035	19	11
Linear SVM	0.982	0.938	0.916	0.927	0.994	0.015	8	11
Random Forest	0.977	0.973	0.840	0.902	0.994	0.021	3	21
Soft-vote ensemble (mean of 4)	–	0.967	0.885	0.924	0.995	–	4	15

4. Performance across subgroups

Key audit finding: The errors are not spread evenly across message types. False positives are more common in short, plain-language conversations: Logistic Regression has precision of 0.08 for short messages and 0.21 for plain messages, compared with 0.99 for long messages. Missed spam is more common in medium-length messages, slang, and adult or premium-rate messages; Naive Bayes recall is 0.70 for slang, while Random Forest recall is 0.33 for medium-length messages.

Subgroup (definition)	n	Spam n	NB recall	LR recall	SVM recall	RF recall
All test messages	1034	131	0.80	0.92	0.92	0.84
Short (≤ 42 characters)	353	2	0.00	0.50	0.50	0.00
Medium (43–91 characters)	346	12	0.50	0.67	0.67	0.33
Long (> 91 characters)	335	117	0.85	0.95	0.95	0.91
URL or 5+ digit number	107	107	0.86	0.96	0.96	0.90
Currency or digits	252	125	0.82	0.93	0.93	0.86
ALL-CAPS heavy ($> 30\%$)	44	19	0.79	0.95	0.95	0.84
Casual / slang	340	37	0.70	0.89	0.89	0.84
Plain (none of the above)	539	6	0.50	0.67	0.67	0.50

The groups overlap because they are defined using simple heuristic rules. The short (2 spam), plain (6 spam) and medium (12 spam) groups are too small for firm conclusions. All four

models miss 11 of the 131 spam messages (8%), and at least one model misses 30. At least one model flags 20 legitimate (ham) messages, although no ham message is flagged by all four models.

5. Explainability check

Method 1: top features (NB log-probability ratios, LR and SVM coefficients, and signed RF feature importance) show that the models rely on similar clues. Terms associated with spam include *txt*, *call*, *text*, *mobile*, *free*, *reply*, *claim*, *prize*, while *ok*, *home* and *later* are more associated with ham. The SVM also placed weight on the topic-specific word *arsenal*, which may indicate overfitting to particular campaigns rather than learning broader spam patterns.

Method 2: failure analysis. Logistic Regression sometimes flags ordinary messages, including “Call me when u’re done...” ($p = 0.55$) and “Where are you call me.” (0.95). SVM assigns “K k:) sms chat with me.” a score of 0.97. Random Forest also flags messages such as “I’m at home. Please call,” possibly because it responds to words like *call* / *service*. The 26 spam messages missed by Naive Bayes had a median predicted probability of just 0.14. Overall, harmless messages containing “call,” “text” or “free” may be flagged, while spam that uses different wording can be missed.

Key finding: preprocessing bug. The `clean_text` function first converts messages to lowercase but then inserts URL, CURRENCY and NUM placeholders in uppercase. The next regular expression `[^a-z\s]` removes uppercase letters, so those placeholders are deleted before the model sees them. For example, “Call 09099726395 now at www.x.com for £100” is reduced to just “call.” In a test version that used lowercase placeholders (the submitted app itself has not been changed), every model performed better:

Model	Prec	Recall	F1 (before → after)	ROC-AUC	FP	FN
Multinomial Naive Bayes	1.000	0.840	0.890 → 0.913	0.992	0	21
Logistic Regression	0.947	0.947	0.889 → 0.947	0.997	7	7
Linear SVM	0.992	0.947	0.927 → 0.969	0.998	1	7
Random Forest	1.000	0.916	0.902 → 0.956	0.998	0	11

6. Limitations and ethical considerations

- **Small audit sample:** The test set contains only 131 spam messages, and the evaluation uses just one random split. As a result, subgroup results may vary considerably, and confidence intervals are not available.

- **Dated, UK-centric, English-only data:** The dataset does not reflect many current scams. People who send short, everyday requests such as “call me” or “are you free” are also more likely to be flagged incorrectly.
- **No word order or context:** The models do not capture a sentence’s word order or full meaning. A word often found in spam can therefore count against an otherwise harmless message, while spam that avoids familiar promotional words may go undetected.
- **Heuristic subgroups, proxy explanations:** The subgroups are based on overlapping regular-expression rules, and the explanations are indirect signals specific to each model. They are not formal SHAP or LIME explanations.
- **Preprocessing issue** (described in Section 5): the current bug removes URL, currency and number cues before they can be used by any of the four models in the submitted app.

7. Recommendations

- **Fix the placeholder bug first** by using lowercase tokens such as urltoken, currencytoken and numtoken. This improves all four models, with the largest F1 gain for Linear SVM (from 0.927 to 0.969) and just one false positive.
- **Choose by cost of error:** Use Linear SVM as the default because the ensemble does not improve on its F1 score. Naive Bayes may be a better fit when false alarms are unacceptable, while Logistic Regression may be preferable when missing spam is the greater concern.
- **Keep a human in the loop,** and retrain the model on newer, more varied messages before real-world use. Then repeat the audit with more examples in each subgroup.